

TaskFlow - Session 3 Project Guide

Fetching Data from an API and Updating the Page

In Session 1 we built the page with HTML and CSS.

In Session 2 we used JavaScript to show our own task data on the page.

In Session 3 we stopped writing the tasks ourselves. Now we ask another computer for them.

Keep this guide. You can read it at home to remember what we did.

Everything here comes from the real project files. Nothing is invented.

1. Where Did We Start Today?

Before today, TaskFlow already worked.

From Session 1

- The header, the three statistic cards, the filter buttons and the task list.
- All the CSS that makes them look like TaskFlow.

From Session 2

- The `tasks` array, written by hand inside `js/app.js`.
- `updateStats()` counted the tasks and wrote the three numbers.
- `getVisibleTasks()` chose which tasks to show.
- `renderTasks()` built the task rows.
- `setFilter()` and the three click listeners made All / Completed / Pending work.

So the page was already alive. But every task was still typed by us.

Today we changed **one thing**: where the data comes from.

2. Session 3: Why an API?

Look at how Session 2 started:

```
const tasks = [  
  { id: 1, title: "Design the TaskFlow page", completed: true },  
  { id: 2, title: "Write the HTML structure", completed: true },
```

```
// ... 4 more  
];
```

Those tasks were **ours**. We typed them.

That is fine for practice, but real applications do not work like this. Your tasks, your messages and your orders are not written inside the web page. They live on a server, and the page asks for them.

Today TaskFlow does the same thing.

3. What Is an API?

An **API** is an address you are allowed to ask for data.

Our page asks. Another computer answers.

```
TaskFlow (our page)  
  |  
  | request ("please give me the tasks")  
  v  
API  
  |  
  | response (the data)  
  v  
TaskFlow (our page)
```

Three words to remember:

- **client** - our page, the one that asks.
- **server** - the other computer, the one that answers.
- **request** and **response** - the question and the answer.

That is all you need for today.

4. The API We Use

This is the real address in our project:

```
const API_URL = "https://jsonplaceholder.typicode.com/todos?_limit=15";
```

`jsonplaceholder.typicode.com` is a free practice API. It exists so people can learn.

`/todos` is the part that says "we want to-do items".

`?_limit=15` asks for only 15 of them. Without it we would get 200.

You can open that address in your browser right now and look at the answer. Do it - it makes everything after this easier to understand.

Why is the address in a `const` at the top of the file?

Because it is written once and never changes while the page runs. Keeping the limit inside `API_URL` makes it easy to change how many tasks we request: you edit one number, in one place, and nothing else in the file has to change.

5. What the API Sends Back

The API answers with a **list**. Each item looks like this:

```
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

Look carefully. You already know three of these:

- `id` - a number for the task.
- `title` - the text of the task.
- `completed` - `true` or `false`.

There is one extra field, `userId`, which we simply do not use. That is normal. An API often gives you more than you need.

The titles are in strange Latin words. That is fine - it is practice data.

6. Same Shape, Different Source

This is the most important idea of Session 3.

Session 2 data:

```
{ id: 1, title: "Design the TaskFlow page", completed: true }
```

Session 3 data (from the API):

```
{ "userId": 1, "id": 1, "title": "delectus aut autem", "completed": false }
```

They have the **same shape**: `id`, `title`, `completed`.

And the whole answer is still an **array of objects** - exactly what we learned in Session 2.

That is why almost none of our old code had to change. JavaScript does not care whether the array was typed by us or downloaded from the internet.

7. `let tasks = []` - Why It Changed

In Session 2:

```
const tasks = [ ... ];
```

The data was there from the very first line.

In Session 3:

```
// The tasks we received from the API. The list starts empty.  
let tasks = [];
```

Two things changed, and both matter.

1. It starts empty.

When the page opens we do not have the tasks yet. We have to ask for them first. So `tasks` begins as an empty array.

2. It is `let`, not `const`.

Later, when the answer arrives, we do this:

```
tasks = await response.json();
```

That gives `tasks` a completely new value. `const` does not allow that. If you try, the browser says:

```
Assignment to constant variable.
```

So the rule from Session 2 still holds: use `const` by default, and `let` when the value will be replaced.

8. The Loading Message and the Error Message

Asking another computer takes time, and it can fail. So the page needs two new messages.

We added them to `index.html` :

```
<!-- Shown while we wait for the API to answer. -->
<p class="message" id="loadingMessage">Loading tasks...</p>

<!-- Shown only when the tasks could not be loaded. -->
<p class="message error hidden" id="errorMessage">
  We could not load the tasks. Please check your internet connection and
  refresh the page.
</p>
```

Notice:

- The **loading** message is the one the user should see first. When the page opens, we really are waiting.
- The **error** message starts hidden, because at the beginning there is no error.

A small difference between the two groups. One group left the loading message visible in the HTML and let JavaScript hide it later; the other gave both messages the `hidden` class in the HTML and let `showLoading()` reveal the loading message. Both end up on the same screen, because the first thing `loadTasks()` does is call `showLoading()` . If your file looks slightly different from the snippet above, that is why - it is not a mistake.

Why bother? Because an empty page tells the user nothing. "Loading tasks..." says *wait, it is coming*. The red message says *something went wrong*. A page that stays silent looks broken.

9. The `hidden` Class

This is the new CSS from Session 3:

```
.message {
    color: #6b7280;
    margin-top: 20px;
    text-align: center;
}

.message.error {
    background-color: #fdeaea;
    border: 1px solid #d64545;
    border-radius: 10px;
    color: #d64545;
    padding: 15px;
}

/* Anything with this class is not shown on the page. */
.hidden {
    display: none;
}
```

`.message.error` (no space) means an element that has **both** classes - the same rule you learned in Session 1 with `.task-status.completed`.

`.hidden` is a small helper class. Any element that has it disappears.

10. Showing and Hiding with `classList`

JavaScript never sets colours here. It only adds or removes the `hidden` class:

```
// Show the waiting message and hide any old error message.
function showLoading() {
    loadingMessage.classList.remove("hidden");
    errorMessage.classList.add("hidden");
}

function hideLoading() {
    loadingMessage.classList.add("hidden");
}
```

```
// Hide the waiting message and tell the user that something went wrong.
function showError() {
  loadingMessage.classList.add("hidden");
  errorMessage.classList.remove("hidden");
}
```

This is the same `classList` idea we used for the blue `active` button in Session 2: **CSS decides how it looks. JavaScript decides which class is on the element.**

Notice that `showLoading()` also hides the error message. That way, if you load again after a failure, the old red message does not stay on the screen.

This is the other small difference between the two groups: one group's `showLoading()` has that

`errorMessage.classList.add("hidden")` line and the other's does not. If yours does not have it, the red message can stay on screen after a later successful load. Adding that one line fixes it.

11. `fetch()` - Asking for Data

`fetch()` is the function that sends the request:

```
const response = await fetch(API_URL);
```

In words: *go to this address and ask for the data.*

`fetch()` does not give you the tasks straight away. It gives you a **response** - which we look at in section 13.

12. `async` and `await`

Our function starts with a new word:

```
async function loadTasks() {
```

`async` means: this function does something that may take time.

`await` means: wait here until this step is finished, then continue.

```
fetch starts
  |
  v
wait
  |
  v
response arrives
  |
  v
continue
```

The rule is simple: you may only use `await` inside a function marked `async`. That is why `loadTasks` has `async` in front of it.

If you forget `await`, JavaScript does not wait. It runs the next line immediately, before the answer exists - and you end up with nothing on the page.

13. `response` and `data` - Not the Same Thing

This confuses almost everybody at first, so read it slowly.

```
const response = await fetch(API_URL);

tasks = await response.json();
```

response is the *envelope*. It tells us whether the request worked, and it holds the answer inside it. It is **not** the list of tasks.

response.json() opens the envelope and turns the text inside into real JavaScript values - an array of objects.

tasks is the result: the array we can finally use.

```
fetch() -> response    (the envelope)
          |
          response.json()
          |
          v
          tasks        (the array of task objects)
```


`response.json()` also takes a moment, so it needs `await` too. That is why there are **two** `await` words in this function, not one.

14. `try` and `catch`

```
try {  
    // the code that might fail  
} catch (error) {  
    // what to do when it failed  
}
```

`try` means: attempt this.

`catch` means: if it failed, run this instead.

If a line inside `try` fails, JavaScript stops there and jumps straight to `catch`. Every line after the failure is skipped.

```
    TRY  
    |  
    fetch the data  
    |  
    did it work? --- yes ---> show the tasks  
    |  
    no  
    |  
    v  
    CATCH  
    |  
    show the error
```

In our project, `catch` runs when the request fails in a way that **throws** - for example a network failure, when the browser cannot reach the server at all.

Test it: turn off your wifi and refresh the page.

15. The Whole `loadTasks()` Function

Now that you know every piece, here is the real function:

```
async function loadTasks() {
  showLoading();

  try {
    const response = await fetch(API_URL);

    // The answer arrives as text, so we turn it into JavaScript objects.
    tasks = await response.json();

    hideLoading();
    updateStats();
    renderTasks();
    updateProgressText();
  } catch (error) {
    showError();
  }
}
```

Read the last four lines inside `try` again:

```
hideLoading();
updateStats();
renderTasks();
updateProgressText();
```

Every one of those is a **function we already had**. `updateStats()` and `renderTasks()` are from the Session 2 class work, and `updateProgressText()` is from the Session 2 homework. We did not rewrite any of them. We just call them once the data has arrived.

Notice also that `showLoading()` is **outside** the `try`. It cannot fail, so it does not need protection. Putting only the risky work inside `try` tells the next reader exactly what you were worried about.

16. Why the Data Arrives Late

Normal JavaScript is extremely fast. Counting six tasks happens instantly.

But asking another computer is different. The request has to travel over the internet, the server has to answer, and the answer has to travel back. That can take a moment - or a long moment on a slow connection.

This is the whole reason Session 3 looks different from Session 2:

Session 2

```
page opens -> data is already here -> render
```

Session 3

```
page opens -> ask -> wait -> data arrives -> render
```

Everything new today - `async`, `await`, the loading message, `try / catch` - exists because of that waiting step.

17. `updateStats()` Still Works

We did not touch this function:

```
// Count the tasks and write the numbers into the three cards.
function updateStats() {
  let completed = 0;
  let pending = 0;

  for (const task of tasks) {
    if (task.completed) {
      completed++;
    } else {
      pending++;
    }
  }

  totalCount.textContent = tasks.length;
  completedCount.textContent = completed;
  pendingCount.textContent = pending;
}
```

It loops over `tasks` and reads `task.completed`. The API tasks have a `completed` property, so the loop works exactly as before.

`tasks.length` is now 15 instead of 6 - and the Total card shows 15 without us changing a single character.

18. `renderTasks()` Still Works

```
for (const task of visibleTasks) {
  let statusClass = "pending";
  let statusText = "Pending";

  if (task.completed) {
    statusClass = "completed";
    statusText = "Completed";
  }

  html += `
    <li class="task-item">
      <span class="task-title">${task.title}</span>
      <span class="task-status ${statusClass}">${statusText}</span>
    </li>
  `;
}
```

`renderTasks()` asks each task for `task.title` and `task.completed`. Both exist in the API data, so the rows build normally.

Say this out loud, because it is the lesson of the day:

`renderTasks()` does not know or care where the tasks came from.
It only needs an array of objects with `title` and `completed`.

That is what good functions look like. They do one job and do not care about the rest.

What if the API had called it `name` instead of `title`? Then every row would show `undefined`. The API decides the property names, not you - so always look at the real data before you write code that uses it.

19. The Filters and the Search Still Work

This is the real `getVisibleTasks()` in our project:

```
function getVisibleTasks() {
  const visibleTasks = [];

  for (const task of tasks) {
```

```

let matchesFilter = false;

if (currentFilter === "all") {
  matchesFilter = true;
} else if (currentFilter === "completed" && task.completed) {
  matchesFilter = true;
} else if (currentFilter === "pending" && !task.completed) {
  matchesFilter = true;
}

const title = task.title.toLowerCase();
const search = searchText.toLowerCase();
const matchesSearch = title.includes(search);

if (matchesFilter && matchesSearch) {
  visibleTasks.push(task);
}
}

return visibleTasks;
}

```

Two checks, and a task must pass **both** before it is shown:

- **matchesFilter** answers *does this task match the All / Completed / Pending button?*
- **matchesSearch** answers *does the title contain what the user typed?* Both sides are made lower case first, so capital letters do not matter.

The search box came from the **Session 2 homework**. It is already part of TaskFlow, and this section exists to make one point: **it kept working when the data started coming from the API**. Nothing in it had to change.

Why? Because it reads `task.title`, and the API tasks have a `title`. The search never cared where the tasks came from.

The listener that feeds it is also unchanged:

```

searchInput.addEventListener("input", function () {
  searchText = searchInput.value;
  renderTasks();
});

```

Click **Completed** after the API data loads and you will see only the completed ones - out of 15 tasks now instead of 6. Then type in the search box while **Completed** is still selected: both narrow the list at the same time.

`setFilter()` and the three click listeners are unchanged too.

20. Why Did We Write It This Way?

We did this	Why
Put the address in <code>const API_URL</code>	Written once, easy to find and change
<code>let tasks = []</code>	The data arrives later and replaces the value
Put the request inside <code>loadTasks()</code>	One clear job with a name, easy to call again
Used <code>async / await</code>	Reads top to bottom, like normal code
Kept <code>response</code> and <code>data</code> separate	The envelope is not the contents
Used <code>try / catch</code>	The request can fail completely
Showed "Loading tasks..."	An empty page tells the user nothing
Showed a red error message	Better to say "it failed" than to stay silent
Called <code>renderTasks()</code> after the data arrived	Before that, there is nothing to draw
Reused <code>updateStats()</code> , <code>renderTasks()</code> and <code>updateProgressText()</code>	They already work with an array of tasks
Kept fetching separate from rendering	One function gets the data, another shows it

The last two lines are the reason Session 3 was short. Good structure in Session 2 made Session 3 easy.

21. How the Project Changed

Before Session 3

- The tasks were typed inside `js/app.js`.
- `updateStats()` and `renderTasks()` ran immediately when the page opened.
- There was nothing to wait for.

After Session 3

- `js/app.js` contains an address, not tasks.
- `tasks` starts empty and is filled when the answer arrives.
- The page shows "Loading tasks..." first.
- If the request fails, the page shows a red error message.
- `updateStats()`, `renderTasks()` and `updateProgressText()` run **after** the data arrives.
- The list shows 15 tasks from the internet instead of our 6.

Everything else - the HTML, the CSS, the filters, the search, the counting, the progress sentence, the rendering - stayed the same.

22. When the Page Opens, What Happens?

This is the most important section of the guide. Learn this order.

1. The browser loads `index.html` and shows the page. The task list is empty and "**Loading tasks...**" is visible.
2. The browser loads `css/style.css`, then `js/app.js`.
3. `js/app.js` runs from top to bottom. It creates `API_URL`, the empty `tasks` array, `currentFilter`, `searchText`, and finds all the elements with `querySelector()`.
4. The last line of the file runs: `loadTasks();`
5. `loadTasks()` calls `showLoading()`.
6. `fetch(API_URL)` sends the request to the API.
7. JavaScript **waits** at that `await`.
8. The answer comes back and is stored in `response`.
9. `await response.json()` turns the answer into an array of objects.
10. That array is stored in `tasks`.
11. `hideLoading()` removes the waiting message.
12. `updateStats()` counts the tasks and writes 15 / 7 / 8 into the cards.

13. `renderTasks()` builds the rows and puts them on the page.
 14. `updateProgressText()` writes the sentence under the list, for example `7 of 15 tasks completed`.
 15. The user can now click All / Completed / Pending **or type in the search box**, and the page filters instantly - no new request is needed, because the tasks are already in the `tasks` array.
- Step 15 is worth noticing. We ask the API **once**. After that, filtering and searching are fast because the data is already here.
-

23. Session 3 HTML and CSS Changes

Very little changed outside the JavaScript.

In `index.html` we added the two message paragraphs, just above the task list:

```
<p class="message" id="loadingMessage">Loading tasks...</p>

<p class="message error hidden" id="errorMessage">
  We could not load the tasks. Please check your internet connection and
  refresh the page.
</p>
```

(As mentioned in section 8, the two groups arranged the starting `hidden` classes slightly differently. Either arrangement works.)

In `css/style.css` we added `.message`, `.message.error` and `.hidden`.

That is all. No card, button, task row, search box or layout rule changed. The search box and the progress sentence were already there from the Session 2 homework, and neither needed a single edit.

24. Complete JavaScript Walkthrough

Here is the whole file, in order, in small pieces.

Part 1 - The API address

```
// The address we ask for our tasks.
const API_URL = "https://jsonplaceholder.typicode.com/todos?_limit=15";
```


Where the tasks come from. `const` , because it never changes.

Part 2 - The data and the filter

```
// The tasks we received from the API. The list starts empty.
let tasks = [];

// Which filter the user selected: "all", "completed" or "pending".
let currentFilter = "all";

// What the user typed in the search box (from the Session 2 homework).
let searchText = "";
```

`tasks` is empty until the answer arrives. All three are `let` because all three change.

Part 3 - The parts of the page

```
const taskList = document.querySelector("#taskList");
const loadingMessage = document.querySelector("#loadingMessage");
const errorMessage = document.querySelector("#errorMessage");

const progressText = document.querySelector("#progressText");
const searchInput = document.querySelector("#searchInput");

const totalCount = document.querySelector("#totalCount");
const completedCount = document.querySelector("#completedCount");
const pendingCount = document.querySelector("#pendingCount");

const filterAllButton = document.querySelector("#filterAll");
const filterCompletedButton = document.querySelector("#filterCompleted");
const filterPendingButton = document.querySelector("#filterPending");
```

The same `querySelector()` from Session 2. Two are new in Session 3: the loading message and the error message. `progressText` and `searchInput` came from the Session 2 homework.

Part 4 - Showing and hiding the messages

```
function showLoading() {
  loadingMessage.classList.remove("hidden");
  errorMessage.classList.add("hidden");
}

function hideLoading() {
  loadingMessage.classList.add("hidden");
}

function showError() {
  loadingMessage.classList.add("hidden");
  errorMessage.classList.remove("hidden");
}
```

Three tiny functions, each with one job. They only add and remove the `hidden` class.

Part 5 - Getting the data

```
async function loadTasks() {
  showLoading();

  try {
    const response = await fetch(API_URL);

    tasks = await response.json();

    hideLoading();
    updateStats();
    renderTasks();
    updateProgressText();
  } catch (error) {
    showError();
  }
}
```

The heart of Session 3. Ask, wait, convert, store, then show.

Part 6 - Counting (from Session 2, unchanged)

```
function updateStats() {
  let completed = 0;
  let pending = 0;

  for (const task of tasks) {
    if (task.completed) {
      completed++;
    } else {
      pending++;
    }
  }

  totalCount.textContent = tasks.length;
  completedCount.textContent = completed;
  pendingCount.textContent = pending;
}
```

Part 7 - Counting the progress sentence (from the Session 2 homework, unchanged)

```
function updateProgressText() {
  let completed = 0;

  for (const task of tasks) {
    if (task.completed) {
      completed++;
    }
  }

  progressText.textContent = `${completed} of ${tasks.length} tasks completed`;
}
```

Called at the end of `loadTasks()` , once `tasks` is full.

Part 8 - Choosing and showing (unchanged)

`getVisibleTasks()` builds the smaller list - checking the filter **and** the search text, as you saw in section 19 - and `renderTasks()` turns it into HTML rows and puts it inside `#taskList`.

Part 9 - The buttons and the search box (unchanged)

```
function setFilter(newFilter, clickedButton) {
    currentFilter = newFilter;

    filterAllButton.classList.remove("active");
    filterCompletedButton.classList.remove("active");
    filterPendingButton.classList.remove("active");

    clickedButton.classList.add("active");

    renderTasks();
}
```

Plus the three `addEventListener("click", ...)` calls, and the search listener from the Session 2 homework:

```
searchInput.addEventListener("input", function () {
    searchText = searchInput.value;
    renderTasks();
});
```

Part 10 - Starting the application

```
// Start the application.
loadTasks();
```

In Session 2 the last lines were `updateStats(); renderTasks();`.

Now there is only one call, because those functions run **inside** `loadTasks()`, after the data has arrived. Calling them here would draw an empty page.

25. Common Session 3 Mistakes

1. Forgetting `await`

```
const response = fetch(API_URL);      // wrong
const response = await fetch(API_URL); // correct
```

Why: without `await`, `response` holds a promise instead of the answer. **Check:** nothing appears, and `console.log(response)` prints `Promise { <pending> }`.

2. Forgetting `async`

```
function loadTasks() {      // wrong - has await inside
  async function loadTasks() // correct
```

Why: `await` only works inside an `async` function. **Check:** the Console shows a red syntax error about `await`.

3. Forgetting `response.json()`

Why: `response` is the envelope, not the tasks. **Check:** the page shows nothing, or `tasks.length` is `undefined`.

4. Using `const tasks`

```
const tasks = []; // wrong
let tasks = [];   // correct
```

Why: `tasks = await response.json()` replaces the value. **Check:** Assignment to constant variable.

5. Calling `renderTasks()` too early

```
loadTasks();
renderTasks(); // wrong - runs before the data arrives
```

Why: at that moment `tasks` is still empty. **Check:** the list stays blank even after the tasks have arrived.

6. Forgetting to call `loadTasks()`

Why: writing a function does not run it. **Check:** the loading message stays on screen forever and there are no errors.

7. A typo in the API address

Why: the server answers "not found" instead of a list of tasks, so there is nothing to draw. **Check:** open the address in a browser tab and see what it actually returns, and look at the **Network** tab for the request's **Status**.

8. Wrong property name

```
task.name    // wrong – the API uses "title"
task.title   // correct
```

Why: the API chooses the names. **Check:** every row shows `undefined`.

9. `querySelector()` found nothing

Why: the id in the HTML and in the JavaScript do not match exactly. **Check:** `Cannot read properties of null`.

10. The error message never appears

Why: `showError()` is not called in `catch`, or the element still has `hidden`. **Check:** turn off the wifi, refresh, and look at the Console.

26. A Simple Way to Debug Session 3

Nothing appears on the page. Do this in order.

1. **Save the file** and **refresh** the browser.
2. **Open DevTools** with `F12` and go to the **Console**.
3. **Read the first red error.** It gives you the file and the line number.
4. **Check the API address.** Open it in a new browser tab. Do you see a list?

5. **Did the request run?** Add `console.log("loading started")` at the top of `loadTasks()`.
6. **Did the answer arrive?** Add `console.log(response)` after the `fetch` line.
7. **Did the conversion work?** Add `console.log(tasks)` after `response.json()`. You should see an array of 15 objects. Click the little arrow to open it.
8. **Did the drawing run?** Add `console.log("rendering")` at the top of `renderTasks()`.

The Network tab is also useful today. Open DevTools, click **Network**, then refresh. You will see the request to `todos?_limit=15`, its **Status** (200 means fine, 404 means not found), and the answer itself under **Response**.

Remember to delete your `console.log()` lines when you are finished.

27. Can You Explain This?

You should be able to say what each of these lines does, in your own words.

```
const response = await fetch(API_URL);
```

```
tasks = await response.json();
```

```
hideLoading();  
updateStats();  
renderTasks();  
updateProgressText();
```

```
} catch (error) {  
  showError();  
}
```

```
loadTasks();
```

If you can explain all five, you understand Session 3.

28. Quick Review Questions

1. What does `fetch()` do?
2. What is the difference between `response` and `tasks`?

3. What does `await` mean?
 4. Why must the function be marked `async` ?
 5. Why is `tasks` declared with `let` and not `const` ?
 6. Why does `tasks` start as an empty array?
 7. Why do we show "Loading tasks..."?
 8. When does the `catch` block run?
 9. Why can `renderTasks()` work with API tasks?
 10. Why do the filter buttons and the search box still work?
 11. Why is `loadTasks()` the only call at the bottom of the file?
 12. What happens if you click Completed - does the page ask the API again?
 13. Which property would break every row if the API renamed it?
-

29. Answers

1. It sends a request to an address and asks for data.
2. `response` is the envelope - it says whether the request worked. `tasks` is the actual array of task objects, after `response.json()` opened the envelope.
3. Wait here until this step finishes, then continue.
4. Because `await` is only allowed inside a function marked `async` .
5. Because `tasks = await response.json()` gives it a completely new value, and `const` does not allow that.
6. Because when the page opens we have not received the data yet.
7. Because the request takes time. Without a message the page would look broken or empty.
8. When the request fails in a way that throws - for example a network failure, when the browser cannot reach the server at all.
9. Because the API tasks have `title` and `completed` , which is all `renderTasks()` needs.
10. Because `getVisibleTasks()` reads `task.completed` and `task.title` , and the API tasks have both properties.
11. Because `updateStats()` , `renderTasks()` and `updateProgressText()` now run inside `loadTasks()` , after the data has arrived.
12. No. The tasks are already in the `tasks` array, so filtering and searching happen instantly with no new request.

13. `title` . If the API called it something else, every row would show `undefined` - and the search would break too, because it reads `task.title` .

30. Glossary

API An address you are allowed to ask for data.

client The program that asks for the data. Here it is our page.

server The computer that answers with the data.

request The question our page sends.

response The answer that comes back. It contains the data and information about whether it worked.

JSON A text format for data. It looks like JavaScript arrays and objects.

`fetch()` The function that sends a request to an address.

`response.json()` Turns the text of the answer into real JavaScript arrays and objects.

`async` Marks a function that does something which may take time.

`await` Wait here until this step finishes, then continue.

`try` Attempt this code.

`catch` Run this instead if the code in `try` failed.

loading state What the page shows while it is waiting for data.

error state What the page shows when the data could not be loaded.

`return` Stops a function immediately.

You Are Done

TaskFlow is finished.

You can now:

- ask an API for data with `fetch()` ,
- wait for the answer with `async` and `await` ,
- turn the answer into JavaScript data with `response.json()` ,
- show a loading message while the user waits,
- handle a failed request with `try` / `catch` ,

- and reuse the functions you already wrote in Session 2 - including the search box and the progress sentence from the homework, which kept working without a single change.

Look back at what happened across the three sessions:

- **Session 1** - HTML and CSS built what you see.
- **Session 2** - JavaScript took data and controlled what appears.
- **Session 3** - the data came from another computer.

The most valuable thing you learned today is not `fetch()`. It is this:

We changed where the data comes from, and almost nothing else had to change.

That happened because each function had one clear job. Keep writing code that way.